Funcionamento e Análise do Algoritmo Merge Sort

1. Visão Geral e Paradigma

O Merge Sort é um algoritmo clássico de ordenação baseado no paradigma de Divisão e Conquista.

Em vez de tentar ordenar o vetor completo de uma única vez, o algoritmo divide recursivamente o problema maior em subproblemas menores. A estratégia funciona em três etapas fundamentais:

- **Divisão:** O vetor original é dividido ao meio sucessivamente até que restem apenas subvetores de tamanho 1.
- **Conquista:** Um subvetor contendo um único elemento é considerado ordenado por definição, portanto não requer ação adicional.
- **Combinação (Intercalação):** Os subvetores adjacentes já ordenados são combinados (*merged*) de forma estruturada para formar subvetores maiores também ordenados, até reconstruir a lista original inteiramente organizada.

2. Funcionamento Detalhado da Implementação

No código do projeto, o Merge Sort opera sobre um vetor de ponteiros para a estrutura Pessoa (Pessoa **arr). Essa abordagem é vantajosa pois evita a movimentação pesada de instâncias inteiras da estrutura na memória, manipulando apenas os seus endereços de memória.

A solução é composta por duas funções que atuam de forma coordenada:

A. Função Recursiva (mergeSort)

Esta função é responsável por gerenciar a divisão do vetor:

1. **Condição de Parada:** A recursão interrompe sua execução quando o índice inicial l não for menor que o índice final r ($l \geq r$). Isso indica que o segmento possui 1 ou 0 elementos.
2. **Cálculo do Ponto Médio:** O índice intermediário é calculado por $m = l + \frac{r-l}{2}$. Esta fórmula evita estouro numérico (*overflow*) de inteiros em cenários com grandes volumes de dados.
3. **Divisão Recursiva:**
    - Chamada para a metade esquerda: mergeSort(arr, l, m, ...)
    - Chamada para a metade direita: mergeSort(arr, m + 1, r, ...)
4. **Intercalação:** Após o retorno de ambos os lados, é invocada a função merge(arr, l, m, r, ...) para unir as duas partes ordenadas.

B. Função de Intercalação (merge)

A função merge realiza o trabalho de comparação e recombinação:

1. **Alocação de Vetores Temporários:** São criados dois vetores auxiliares de ponteiros, chamados L e R. O tamanho de L é $n_1 = m - l + 1$ e o de R é $n_2 = r - m$.
2. **Cópia de Referências:** Os ponteiros das posições correspondentes do vetor principal arr são copiados para L e R.
3. **Comparação Ordenada:**
    - São utilizados três ponteiros de índice: $i = 0$ (para L), $j = 0$ (para R) e $k = l$ (para o segmento em arr).
    - O laço principal compara as chaves de busca (L[i]->rg e R[j]->rg).
    - O ponteiro contendo o menor valor de RG é inserido de volta na posição arr[k], incrementando a posição de k e do subvetor de onde o elemento foi retirado (i ou j).
4. **Esvaziamento de Remanescentes:** Caso um dos subvetores seja totalmente consumido antes do outro, laços adicionais copiam os elementos restantes diretamente para arr.

3. Análise de Complexidade de Tempo e Espaço

O desempenho do Merge Sort apresenta características previsíveis e consistentes:

- **Complexidade de Tempo no Pior Caso:** $O(n \log n)$ Mesmo que a lista esteja em ordem inversamente classificada, a árvore de divisão recursiva terá sempre altura $\log_2 n$, e cada nível exigirá $O(n)$ comparações e movimentações para intercalar os elementos.
- **Complexidade de Tempo no Caso Médio:** $O(n \log n)$ Independente da distribuição dos valores no vetor inicial, o padrão de divisão e intercalação é mantido.
- **Complexidade de Tempo no Melhor Caso:** $O(n \log n)$ Mesmo que a lista já esteja perfeitamente ordenada, o algoritmo realiza todas as etapas de divisão e reconstrução.
- **Complexidade de Espaço Auxiliar:** $O(n)$ O algoritmo necessita de memória adicional proporcional ao número de elementos n para armazenar os vetores auxiliares de ponteiros L e R durante o processo de intercalação.

4. Principais Vantagens no Contexto do Projeto

- **Comportamento Previsível:** Ao contrário do Quick Sort, que pode degenerar para $O(n^2)$ no pior caso se a escolha do pivô for desfavorável, o Merge Sort mantém garantia estrita de tempo $O(n \log n)$.
- **Estabilidade da Ordenação:** Trata-se de um algoritmo estável. Se houver registros com RGs idênticos, a ordem relativa em que eles apareceram originalmente é preservada devido ao teste <=.
- **Otimização por Uso de Ponteiros:** O custo da memória auxiliar $O(n)$ é reduzido significativamente no seu código, pois aloca-se apenas o espaço para armazenar ponteiros simples (Pessoa *), e não as estruturas completas.